

University of London: Goldsmiths - BSc Computer Science

Module: CM3070 - Final Project

Student number: 230462218

April 2026 – September 2026

Preliminary Report

Preliminary Report

8.2 Project Idea 2: Towards a Low-cost Resilient Early-warning Natural Disaster Monitoring, Notification and Rapid Response Platform

4573 Words (excluding headings, tables, and captions)

Introduction: 375 words

Literature Review: 2496 words

Design: 1386 words

Prototype Implementation: 316 words

(word count excludes headings, tables and captions for each section)

Table of Contents

1 – Introduction.....	3
2 - Literature Review	4
2.1 - Review of Existing Communication Technologies and IoT Hardware	4
2.2.1 – Processing Nodes: Single Board Computers (SBCs) vs Microcontroller Units (MCUs).....	4
2.1.2 – Communication Topologies and Networking Protocols	5
2.2 - Review of Existing IoT Natural Disaster Management Solutions	6
2.2.1- IoT-based Flood Early Warning System for Effective Disaster Management (Raman & Iqbal, 2024)	6
2.2.2 – An intelligent IoT–machine learning framework for wildfire detection and prediction using a hybrid RF–XGB model (Radhi & Ibrahim, 2026)	7
2.2.3- IoT Integrated Smart Rover System for Disaster Relief Management (Pragatheeswari et al., 2024)	8
2.2.4 – Synthesis and Conclusion	9
3 – Design.....	12
3.1 - Overview, Domain and Target User Identification	12
3.2 - Design Structure.....	13
3.3 - Justification of Key Design Choices	20
3.4 - Work Plan.....	22
3.5 - Testing and Evaluation Plan.....	23
4 – Feature Prototype	24
References.....	27

1 – Introduction

“Earth’s climate swings are increasingly out of balance” [1] making our atmosphere more prone to extreme weather due to climate change. This imbalance results in a phenomenon known as “hydroclimate whiplash – rapid swings between intensely wet and dangerously dry weather” [2]. Rather than occurring in isolated incidents, current disasters are occurring in compound loops, with one natural disaster feeding into the next – heavy rainfall brings vegetation which is then burned and dried during periods of hot weather causing drought, becoming fuel that later ignites as wildfires [2]. This now baked and dry land becomes hydrophobic, unable to absorb any water, flooding the area during the next period of intense rainfall [2]. Since these disasters no longer occur in isolation, managing them with fragmented tools is not a solution and they must be jointly monitored [2].

Britannica states that “no spot on Earth is immune from natural disaster”, but certain geographic areas may be prone to frequently occurring, specific disasters [3]. Current traditional early disaster warning systems include land-based seismic sensors, stationary buoys placed in oceans, sensors integrated into aircrafts and satellite-based monitoring [3]. However, these industrial-grade solutions are not low cost, making them inadequate for dense deployment in resource-constrained developing countries, or local areas such as isolated, off-grid households. Furthermore, traditional communication methods, such as radios, SMS alerts and television announcements to inform and warn impacted populations depend heavily on centralised vulnerable infrastructure such as cellular masts, landlines and Internet infrastructures. Such dependencies can collapse during intense rainfall, become damaged during wildfires, floods and high heat, therefore cutting-off vulnerable remote residents from emergency services and communication.

To overcome these limitations, recent literature highlights the potential of Internet of Things (IoT) solutions as a paradigm shift in natural disaster monitoring and communication [4, 5, 6]. IoT solutions have the ability to provide predictive analytics, facilitate early warning communication, facilitate rescue operations, and assist in post-disaster healing [5, 6].

This paper will begin by reviewing current state-of-the-art Internet of Things technologies, assess existing IoT solutions for natural disaster monitoring, before proposing an IoT natural disaster management, mitigation and communication platform that targets droughts, wildfires and flooding. This project is based on project idea 8.2 – ‘Towards a Low-cost, Resilient Early-warning Natural Disaster Monitoring, Notification and Rapid Response Platform’.

2- Literature Review

2.1- Review of Existing Communication Technologies and IoT Hardware

Internet of Things (IoT) architectures for natural disaster management should consider computational capability, energy efficiency and network resilience. The following sections review current state-of-the-art technologies to form a baseline understanding to assist in evaluating existing solutions in section 2.

2.2.1 – Processing Nodes: Single Board Computers (SBCs) vs Microcontroller Units (MCUs)

Primary architectural design involves selecting edge processing hardware, which generally falls in two types: Single Board Computers (SBCs) and Microcontroller Units (MCUs) [7]. SBCs, such as the Raspberry Pi, are fully functioning computers in a single circuit, integrating processors, memory, storage input/output (I/O) ports and also have the capability of running an operating system (OS), such as Linux [7, 8, 9]. Furthermore, they have the capability to simultaneously run instructions, and support high-level programming languages and frameworks, allowing them to execute complex tasks at the edge, such as machine learning algorithms [7, 9].

Microcontrollers, on the other hand, such as the Arduino Uno, ESP32 and ESP8266, are designed to execute simpler control tasks by combining a processor, memory, and programmable input/output (I/O) peripherals on a single chip [7]. Unlike multi-tasking SBCs, microcontrollers execute single instructions sequentially, through a continuous deterministic loop, and therefore are restricted to simpler edge processing. However, this architectural simplicity allows MCUs to operate with microsecond responsiveness, maintaining low-power states.

As opposed to microcontrollers, despite the computational power of an SBC seeming ideal, SBC's cannot enter deep-sleep states, creating a constant power consumption that makes long-term battery operation unviable during power grid failures or deployment in remote areas [10]. Consequently, the processing node selected for this project must balance the computational demands of edge processing against maintaining low-energy constraints required for natural disaster monitoring.

Table 1.1 compares the properties of SBCs and Microcontrollers.

	SBCs	MCUs
Examples	Raspberry Pi, NVIDIA Jetson Orin Nano	Arduino Uno, ESP32, ESP8266
OS & Stack	Full OS & Libraries	Bare-Metal Firmware/ light Real-Time Operating System (RTOS)
Power Consumption	High – Processing power and lack of sleep cycles	Low – limited processing and deep sleep state abilities
Edge Processing Capability	High – Machine Learning, data analysis, advanced signal processing	Low to Moderate – basic sensor processing,
Cost	Higher	Low

Table 1.1 – A comparison of Single Board Computers and Microcontroller Properties.

2.1.2 – Communication Topologies and Networking Protocols

According to De Masi [11], usually mesh or star topologies are implemented in IoT solutions; edge processing nodes directly communicate with a main gateway in a star topology, whereas in a mesh topology, edge nodes communicate with nearby nodes, either directly or indirectly. Due to this complex structure, mesh networks are more resilient to failure compared to star topologies since there is no dependency on a central gateway [11]. Consequently, this makes star topologies less reliable in natural disaster monitoring systems, compromising the entire network in the case of operational or hardware failure.

Current commonly used communication technologies in IoT solutions include Wi-Fi, ESP-NOW, and LoRa [12]. ESP-NOW is a recently emerging connectionless wireless protocol defined by Espressif allowing two microcontrollers, particularly ESP32 and ESP8266, to directly communicate peer-to-peer without requiring a Wi-Fi router or gateway hub [12, 13]. While Wi-Fi and ESP-NOW are designed for shorter-range local area networks (LAN), LoRa is intended to be used in wide area networks (WAN) by operating as the physical layer using long-range radio modulation to transmit data [12, 14].

Eridani et al. [12] conducted a study to compare different communication protocols commonly used in IoT solutions, including Wi-Fi, ESP-NOW and Bluetooth. These protocols were tested using an ESP32 microcontroller utilising both its internal antenna, as well as an external antenna in a peer-to-peer communication. Each of these protocols was tested for suitability in different conditions. Their results can be seen in Figure 1.1 [12], indicating that ESP-NOW is more suitable protocol for natural disaster monitoring systems due to its higher-range coverage and lowest latency, despite its high-power usage and transmission speed. However, a significant drawback of this study is that testing was restricted to isolated, peer-to-peer communication, failing to evaluate the performance of how each protocol would handle packet collisions or network congestion when scaled across multiple nodes, which

would likely occur when sensor nodes simultaneously alert changing conditions in an IoT natural disaster management solution.

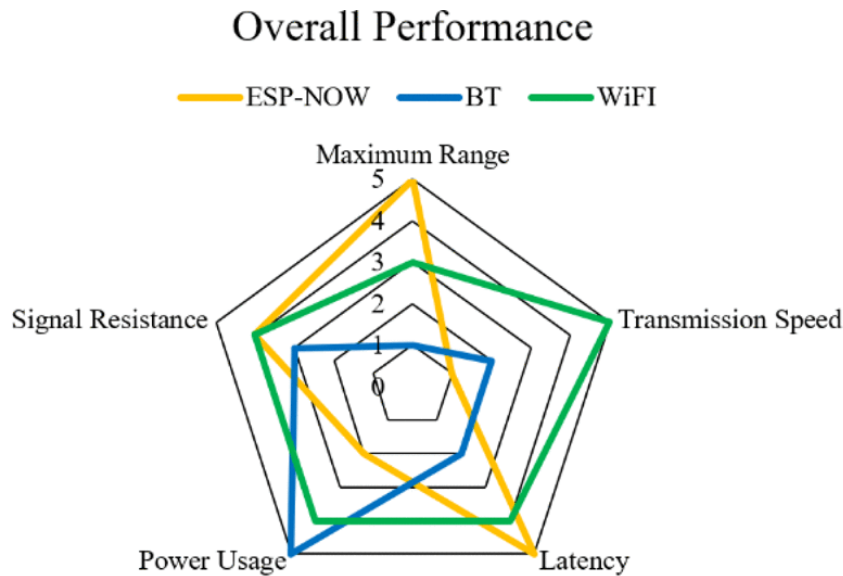


Figure 1.1 – Results from the study conducted by Eridani et al. Taken from [12]

2.2- Review of Existing IoT Natural Disaster Management Solutions

To identify potential implementation strategies for the proposed solution, online literature regarding existing IoT natural disaster management solutions found using search terms including: IoT wildfire monitoring, IoT flood detection, low-cost natural disaster IoT, and resilient IoT for natural disaster were analysed. Three existing implementations were studied as described in the following sections.

2.2.1- IoT-based Flood Early Warning System for Effective Disaster Management (Raman & Iqbal, 2024)

The IoT Flood Early Warning Framework developed by Raman and Iqbal [15] deployed a comprehensive sensor array consisting of an ultrasonic sensor to monitor water levels, an electromagnetic sensor to measure water flow rates, a tipping bucket sensor to monitor rainfall intensity, and temperature and wind dynamic sensors. Moreover, a Raspberry Pi was used as a low-cost, single board computer to aggregate telemetry data received from the

sensors, routing it to a centralised cloud platform via localised Wi-Fi. The cloud platform stored and processed the sensor data alongside past data using machine learning systems to generate flood risk predictions, and consequently alert stakeholders consisting of “government entities, first responders, and the general people” [15, p.2] via a mobile application.

At first glance, the architecture posed by Raman and Iqbal [15] offers several distinct advantages in the domain of natural disaster monitoring. Avoiding reliance on a single factor by integrating multiple flood-related sensors ensures a holistic, multi-factor assessment of the environment, enabling highly accurate reasoning for predicting possible floods. Furthermore, the solution moves away from rigid, hardcoded thresholds and instead adopts machine learning-driven predictions which enables the system to recognise complex historical patterns and deliver proactive, adaptive forecasting tailored to its location, making its predictions more reliable. This predictive intelligence directly enhances public safety by automatically pushing real-time evacuation data through a mobile application, bridging the gap between edge-based data processing and scalable public safety coordination.

However, Raman and Iqbal's [15] selection of a high-overhead Raspberry Pi Single-Board Computer (SBC), rather than a low-power Microcontroller Unit (MCU) like an ESP8266, introduces a major energy deficit. Because the platform lacks hardware-driven low-power deep-sleep states, its continuous high-wattage current consumption makes battery-powered operation unviable during extended grid failures [10]. Additionally, the architecture's strict reliance on localised Wi-Fi and cloud servers imposes a critical system vulnerability - if local telecom towers collapse due to severe weather conditions or a natural disaster for example, critical data becomes trapped at the edge node, leaving residents and authorities severely compromised to imminent threats. Moreover, the choice of communicating alerts exclusively through a mobile application poses considerable limitations, leaving those without the mobile app blind to critical alerts. Furthermore, the solution completely lacks autonomous mitigation capabilities to actively reduce disaster impact, such as autonomous flood gate activation, and only focuses on detecting and communicating findings.

2.2.2 – An intelligent IoT–machine learning framework for wildfire detection and prediction using a hybrid RF–XGB model (Radhi & Ibrahim, 2026)

Radhi and Ibrahim [16] proposed an IoT and machine learning solution for real-time wildfire prediction using a dual-tier machine learning pipeline and Arduino Nano edge boards. To detect pre-combustion conditions, environmental metrics were logged through integrated temperature, humidity (DHT11), and carbon monoxide (MQ-7) sensors. A lightweight Multiple Logistic Regression (MLR) model ran directly on these edge nodes to calculate local wildfire probabilities, enabling the nodes to remain in a deep-sleep state during routine duty cycles. Upon detecting an anomaly or breach to the predefined wildfire probability threshold, through an nRF24L01 Radio Frequency (RF) transceiver the corresponding node communicated directly to the sink node hosting a hybrid Random Forest and Extreme Gradient Boosting (RF-XGB) validation model. Local physical alarms and automated Telegram

bot notifications were then triggered, whilst simultaneously the cloud dashboard and interactive mobile mapping application were updated.

A core strength of this IoT solution lies in its integration of hierarchical machine learning processing. The hybrid machine learning algorithm proposed by Radhi and Ibrahim [16] is shown to outperform traditional machine learning models with over 96% predictive accuracy and preventing false positive alarms. This capability improves early-warning reliability by preventing wasteful emergency deployment whilst providing certainty that an alert signifies a potential upcoming wildfire. Moreover, the solution highlights a strong focus on energy optimisation through its implementation of hourly duty cycling. Nodes remain in a low-power sleep mode, only transmitting data when the probability of a wildfire exceeds a predefined threshold. Consequently, this selective transmission reduces network traffic and allows the nodes to achieve an “analytically estimated lifetime of up to 11 months” [16 p.2], making the architecture highly viable for deployment in remote locations.

Despite this, implementing sleep cycles potentially compromises monitoring resilience; if an edge sensor node experiences a hardware fault or faces structural damage prior to exceeding the threshold during this state, it will remain undetected since the system receives no signal, mistaking node failure for a safe environment. Furthermore, the utilisation of a hierarchical cluster topology introduces serious communication vulnerabilities. Since sensor edge nodes depend on a cluster head sink node to pass data to the base station gateway, the network introduces a single point of failure. Consequently, a failure in this sink node collapses the dependent network, leaving the area unmonitored. This could be bypassed by transitioning to a decentralised peer-to-peer or mesh architecture, for example, where nodes communicate directly to one another, eliminating central dependency. Finally, like the work of Raman and Iqbal [15], the platform’s multi-notification implementation depends upon functional internet infrastructure (Telegram and cloud servers), rendering the alert system to be non-functional if internet connectivity is lost.

2.2.3- IoT Integrated Smart Rover System for Disaster Relief Management (Pragatheeswari et al., 2024)

Pragatheeswari et al. [17] presented an IoT-Integrated Smart Rover for Disaster Relief Management. The rover integrated a Node MCU ESP32 microcontroller and a 12V power supply, alongside an ESP32 Cam and a sensor group consisting of: a HC-SR04 ultrasonic sensor, a MQ2 gas sensor, a DHT11 temperature sensor, a KY-026 flame sensor and a SW-420 vibration sensor. To provide mobility, this hardware was integrated with DC tyres and motors, enabling the prototype to navigate an area for environmental monitoring. Live sensor data and camera feed could be monitored through the cloud-based Blynk mobile application, generating an alert if sensor readings exceeded a defined threshold.

Upon initial review, this solution can be seen to utilise cost-effective, easily available hardware, making it scalable and adaptable to different disaster monitoring domains. Using a low-cost commonly available microcontroller like the ESP32 ensures high modularity, allowing individual connected sensors, and the microcontroller itself, to be easily upgraded

or replaced in the case of failure or damage, ensuring minimised downtime and rapid deployment. Additionally, the conversion of a typical IoT disaster monitoring and relief management solution being implemented as a rover facilitates remote operation and evaluation of the surrounding area through the sensors and live video stream from the ESP32 CAM, allowing a more direct observation and evaluation of environment conditions [17]. This transition from a traditional static IoT solution to a mobile solution allows expands the geographic coverage of the system, reducing the need of deploying additional nodes and sensors in nearby areas. Moreover, the Blynk cloud framework utilised by Pragaswari et al. [17] uses a streaming connection protocol, minimising network overhead, allowing connectivity to be maintained even in disaster-prone areas with remote or challenging connectivity conditions [18].

However, dependency on this cloud infrastructure introduces a severe communication vulnerability since it cannot function if local Wi-Fi or cellular network completely fails, thereby reducing resilience required for an IoT natural disaster monitoring framework. Moreover, continuously running an ESP32 Cam with the sensors and DC motors may likely create a high-power processing and energy consumption, severely limiting the rover's operational run time, causing latency to the camera feed when simultaneously reading and transmitting data from the sensors.

2.2.4 – Synthesis and Conclusion

Collectively evaluating existing IoT-based solutions for natural disaster management and monitoring across floods, wildfires and broader environmental monitoring, displays diverse approaches to low-cost solutions produced with easily accessible hardware. All of the reviewed solutions provide multi-factor environmental assessments, rather than depending on a single factor to evaluate conditions and alert when necessary. This all-rounded approach contributes towards a more holistic, and thus informed, justified prediction, and therefore is something the solution personally proposed should consider.

Furthermore, the framework proposed by Raman and Iqbal [15] utilises a Raspberry Pi Single Board Computer, as opposed microcontrollers which Radhi and Ibrahim [16] and Pragaswari et al. [17] can be seen to have used. As previously critiqued, this selection of an SBC introduces a severe energy deficit in comparison the microcontroller-based (MCU) approaches of Radhi and Ibrahim [16] and Pragaswari et al. [17]. However, within these solutions, only the wildfire prediction solution developed by Radhi and Ibrahim [16] addresses the issue of energy efficiency and robustness through their implementation of having sensor-edge nodes enter sleep states and hourly reading cycles. This is a highly effective technique that the personal solution proposed should implement to ensure maximum operational longevity and energy efficiency which is vital for natural disaster monitoring, particularly in remote areas.

It can also be seen that the flood monitoring framework proposed by Raman and Iqbal [15] and the wildfire prediction solution developed by Radhi and Ibrahim [16] both integrate machine learning models into their solution to assist in reliable prediction and reduce false

predictions, whereas Pragatheeswari et al. [17] use pre-defined thresholds to alert users of conditions. Utilising machine learning algorithms, especially when combined with historic readings or data, initially seems ideal; however, the complexity of the model that can be used is dependent on the embedded computing platform utilised. Despite Raman and Iqbal's [15] framework being critiqued for high energy consumption, using the Raspberry Pi SBC allows for more advanced levels of machine learning models to be applied. In contrast, the Arduino Nano microcontrollers forced Radhi and Ibrahim's [16] solution to use a simplified version of their Multiple Logistic Regression model despite being previously praised for energy efficiency. Consequently, comparing these solutions demonstrates the compromise in computational performance and energy efficiency. To address this balance, the solution proposed will implement fuzzy logic to evaluate environmental conditions and generate alerts. This approach transitions away from utilising predefined thresholds, as implemented by Pragatheeswari et al. [17], whilst still making logical evaluations requiring less power and energy than machine learning algorithms, and thus being able to be implemented on an energy-efficient microcontroller.

Moreover, all three solutions display a main reliance on internet connectivity - cellular, Wi-Fi or cloud-based - for communication and networking, making them less resilient for natural disaster monitoring. This dependency makes the frameworks vulnerable in cases of internet infrastructure failure or damage, or difficult to deploy in remote areas with weak transmission signals. Additionally, all of the solutions can be seen to use mobile applications to provide warnings, with Raman and Iqbal [15] using a mobile application they have developed, Radhi and Ibrahim [16] relying on Telegram notifications and Pragatheeswari et al [17] utilising a cloud-based Blynk mobile app, making the assumption that relative parties will have downloaded the app. Based on this, the proposed solution will treat Internet-based communication as a 'luxury'/ second enhancement, instead relying on ESP-NOW based communication, only actively communicating to the dashboard when wider area networks are functional, defaulting to direct peer-to-peer when not possible.

Finally, it can be seen that all three solutions only act as monitoring and detecting platforms, focusing on communicating and do not actively attempt to mitigate the impact of the disaster apart from alerting. The proposed solution considers this gap by actively taking necessary actions when it detects possibilities of a natural disaster occurring, such as watering soil to increase soil moisture upon detecting pre-drought or pre-wildfire conditions.

Table 2.1 compares the reviewed solutions in this section, alongside what is personally proposed.

	2.1 – Flooding [15]	2.2 – Wildfire [16]	2.3 – Rover [17]	Proposed Solution
Core Hardware (Processing Unit)	Raspberry Pi – SBC (Single Board Computer)	Arduino Nano - Microcontroller	ESP32 - Microcontroller	ESP8266 - Microcontroller
Multiple-sensor Reading	Yes	Yes	Yes	Yes
Sensors Used	Ultrasonic (Water Level), Electromagnetic (Flow Rate), Tipping Bucket (Rainfall), Digital (Temperature/ Humidity), Cup Anemometer (Wind Speed/Direction)	temperature, humidity (DHT11), and carbon monoxide (MQ-7) sensors	HC-SR04 ultrasonic sensor, MQ2 gas sensor, DHT11 temperature sensor, KY-026 flame sensor and SW-420 vibration sensor.	Integrated Multi-Hazard Sensor Array (ultrasonic sensor (water level), DHT11 temperature and humidity sensor, soil moisture sensor and weather APIs).
Decision-Making	Machine Learning and Historical Data	Machine Learning – Simplified Multiple Logistic Regression	Predefined Thresholds	Fuzzy Logic with Historical Data and Forecasts
Network Protocol	Local Wi-Fi and Cloud	nRF24L01 RF (Radio Frequency), Cloud and Telegram	Blynk Cloud & Wi-Fi/Cellular	ESP-NOW (main) and Wi-Fi (secondary)
Network Topology	Wi-Fi to Centralised Cloud	Hierarchical Cluster	Centralised Cloud Connection	Decentralised Peer-to-Peer
Alert/ Notification Communication	Mobile Application	Local Physical Alarms, Automated Telegram Bot Notifications, Cloud Dashboard, Interactive Mobile Mapping Application	Blynk Mobile Application	Web Dashboard and Local Physical Alarms
Active Mitigation Capability	None – Detection and Communication Only	None - Detection and Communication Only	None - Detection and Communication Only	Yes – Automated Edge Actions/ Hardware Edge Actuators (floodgate and irrigation)

Table 2.1 – A comparison of the solutions presented by [9], [10] and [11], alongside what is proposed in this paper

3 – Design

3.1- Overview, Domain and Target User Identification

After analysing literature, it can be concluded that an ideal solution should incorporate multiple sensors, data-driven reasoning combined with artificial intelligence, such as machine learning or fuzzy logic, and be able to actively mitigate. Due to the extreme environment in which natural disaster monitoring solutions operate, an ideal solution should be robust, resilient to power failures, Internet outages, external damage, and be power efficient. The proposed project aims to deliver a low cost IoT platform developed to detect, monitor and mitigate cascading natural disasters (flooding, drought and wildfire) and be resilient to power-constraints and network infrastructure failures. It will integrate numerous sensors at an edge node to analyse environment conditions, also considering weather forecasts to evaluate the area condition using embedded fuzzy logic inference. The platform will autonomously scale its operational duty cycles, alert and take necessary actions through mitigation and alert nodes if it evaluates pre-natural disaster conditions. Nodes will operate in a decentralised manner independent to vulnerable network infrastructure, communicating primarily through ESP-NOW, only using Wi-Fi if needed and available. A dashboard will also be developed to monitor nodes and visualise them on a map, view sensor history and control them, request ‘force’ mitigation actions and provide Twilio access to ‘force’ a community alert.

This project is based on project idea 8.2 – ‘Towards a Low-cost Resilient Early-warning Natural Disaster Monitoring, Notification and Rapid Response Platform’. The domain of this project is environmental monitoring, natural disaster mitigation, and resilience engineering. Intended target users include:

- Local municipalities and emergency authorities – regional authorities requiring highly-localised environmental insights and a dynamic policy dashboard to coordinate actions before, during and post-disaster.
- High-risk, disaster-prone resident demographics – individuals and communities living in geographically vulnerable zones, such as those living in floodplains, shorelines, hot climates, etc. They require immediate local alerts that still function in pre- or during disaster conditions.
- Resource-constrained developing countries – regional authorities that cannot afford traditional, industrial-grade natural disaster management tools, thus requiring low-cost, accessible solutions that allows for dense deployment.
- Vulnerable remote communities and individual homeowners – off-grid or isolated households and communities situated far from immediate emergency services who require localised, automated platform capable of alerting and taking action if they are cut access to closest areas.

The aims and objectives of this project can be seen in table 3.01.

Aims	Implement a low-cost IoT natural disaster monitoring, notification and mitigation solution
	Implement data-driven reasoning directly at the edge to allow environments to be assessed locally
	Implement a resilient wireless communication architecture without reliance on external network dependencies
	Implement an IoT solution that optimises power efficiency and ensures long-term field survival
	Implement a dashboard that allows authorities to define policies and manage the platform
	Implement a resilient connection gateway between decentralised sensing network and public communication systems
Objectives	Construct a low-cost multi-tier hardware network architecture using accessible microcontrollers and sensors to detect, mitigate and communicate environmental status
	Implement an edge-optimised fuzzy-logic inference system to evaluate environment conditions
	Develop a decentralised, resilient wireless communication framework to achieve rapid, robust data transmissions directly between sensor arrays and mitigation actuators primarily using ESP-NOW in a mesh topology.
	Develop an adaptive control algorithm using fuzzy-logic and weather forecasts that dynamically alters sensor reading frequencies and power sleep modes to maximise energy efficiency
	Integrate predictive weather forecast API's for reliable predictions and automated public alert API's to facilitate emergency notifications
	Develop a dashboard to visualise sensor status, manage sensor and mitigation nodes, allow authorities to create policies and override current state, and allow 'force' actions

Table 3.01 – Aims and Objectives of the proposed project

3.2- Design Structure

The IoT solution is organised in a modular, three-tier topology separating sensor nodes, mitigation nodes and alerting nodes, designed to eliminate single points of failure during natural disasters. These three nodes are constructed using an ESP8266 microcontroller; this selection is justified by its financial viability for budget-constrained authorities, its sleep state abilities to ensure power-optimisation and built-in Wi-Fi capabilities for secondary communication, making it an ideal choice for resilient IoT natural disaster monitoring platforms.

The sensor node operates as the primary layer to evaluate environmental conditions and acts as the local processing unit of the edge network. Connected sensors – ultrasonic for water level monitoring, DHT11 for temperature and humidity monitoring and soil moisture level sensor – take readings of the environment periodically at intervals defined by the system administrator (e.g. local authorities) through the main dashboard. These readings are then processed at the edge through an embedded fuzzy logic engine, combined with local weather forecast data from an API to calculate current conditions and risk levels. This allows the node to immediately process sensor data and pass it on to the dashboard whilst simultaneously updating the alarm node with the current environmental assessment status. If pre-natural disaster (wildfire, drought, flood) or critical conditions are evaluated, then the node automatically dispatches an execution command to the mitigation node using ESP-NOW to either activate the floodgate or the irrigation system, or both, as well as updating the dashboard and the alarm node. Sensor reading intervals are adapted to monitor more frequently based on the seriousness of the evaluation. When sensors are not taking a reading or their data is being processed, this node enters a sleep state, as to conserve power, waking up for the next reading.

The key components, their roles and requirements for node 1 (sensor node) can be seen in table 3.3.1 below, and the circuit diagram can be seen in figure 3.3.1 below.

Node 1 – Sensor Node				
Component Image	Component Name	Component Description	Role	Requirements
	ESP8266 Microcontroller	A WiFi enabled microcontroller	Used to take sensor readings at timely adaptive intervals, evaluate environment conditions using fuzzy logic, in deep sleep state when not monitoring or evaluating	3.3V Power source
	Ultrasonic Ranging Module	A sensor that calculates the distance of objects from its position	Used to monitor water levels for flood monitoring, high levels = flood conditions	N/A
	DHT11 Temperature and Humidity Sensor	A sensor that monitors temperature and humidity	Used for all three types of natural disaster monitoring and pre-condition detecting	10K resistor

			• Wildfire = high temperature and low humidity • Drought = high temperature and low humidity • Flood = high humidity	
	Capacitive Soil Sensor	A sensor that monitors soil moisture levels, electrodes are not exposed in capacitive soil moisture sensors so resistant to corrosion	Used for all three types of natural disaster monitoring and pre-condition detecting: • Wildfire = low soil moisture • Drought = very low/ deficit moisture • Flood = high/ saturated moisture	N/A

Table 3.2.1 – Component breakdown for Node 1 (Sensor Node)

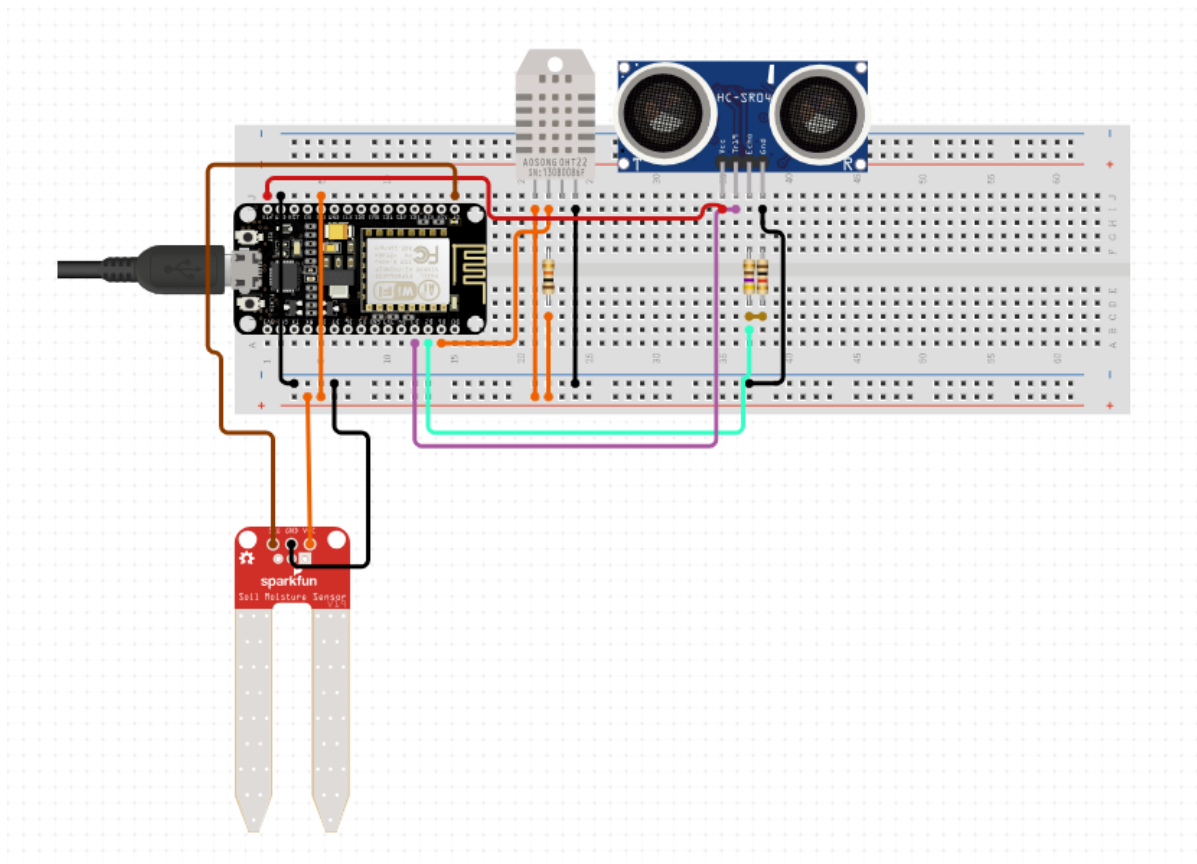


Figure 3.2.1 – Circuit Diagram for Node 1 (sensor node)

Node 2, the mitigation node, acts as the defence mechanism of the edge architecture. It waits for instructions from a sensor node or the dashboard to activate or deactivate the connected floodgate or irrigation system. Thus, sleep cycles are not implemented in this node due to it having to actively monitor and accept incoming instruction messages.

This mitigation node consists of an ESP8266 microcontroller, a servo motor to act as the control for the floodgate, and an irrigation system consisting of a water pump and vinyl tubing. The breakdown and role of these components can be seen in table 3.2.2, and the corresponding circuit diagram can be seen in figure 3.2.2.

Node 2 – Mitigation Node				
Component Image	Component Name	Component Description	Role	Requirements
	ESP8266 Microcontroller	A WiFi enabled microcontroller	Listens for alerts and activates floodgates and irrigation system based on communication	3.3V Power source
	Servo	A component that rotates its shaft to control the angular position of an object connected to it	Control the floodgate position (active/not active) depending on environment conditions	Libraries: • Servo.h – native Arduino library for servo
	Water Pump with Vinyl tubing (irrigation system)	A component that will move water from a source to through the vinyl tubing	Water the ground to increase soil moisture levels to prevent wildfire and drought conditions	• 5v Relay Module

Table 3.2.2 - Component breakdown for Node 2 (Mitigation Node)

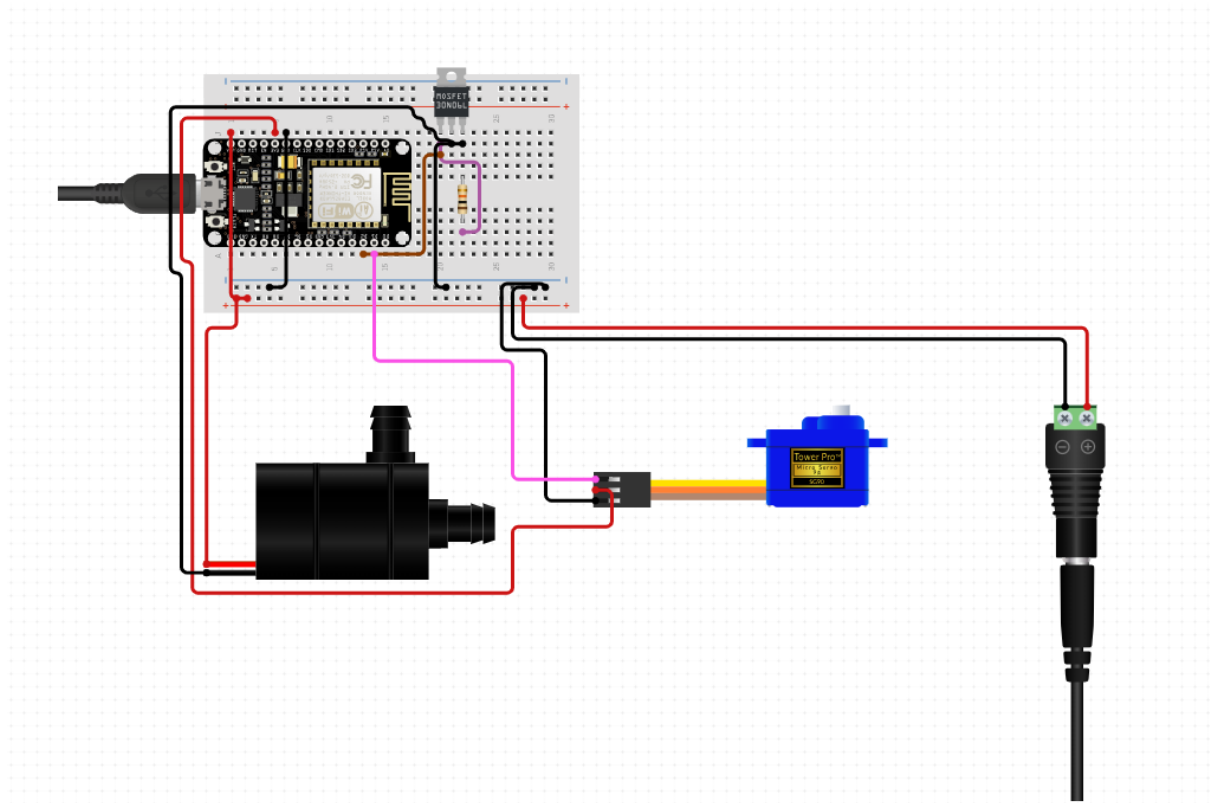


Figure 3.2.2 – *Circuit Diagram for Node 2 (mitigation node)*

The alerting node (node 3), serves as the public information and warning node for this solution, ideally positioned in residential areas, or around vulnerable boundaries to deliver immediate warnings and evacuation alerts. Its corresponding ESP8266 microcontroller actively listens for any packets sent by the sensor node, and thus, similar to node 2, it doesn't utilise sleep cycles. LEDs light up depending on current condition status in a traffic light manner, whilst the LED screen displays a message regarding the evaluated environment status. Depending critical conditions, an alarm is sound through the buzzer depending on the severity. This node also functions as a local webserver, utilising the built-in Wi-Fi capabilities of the ESP8266, to host an emergency status page without an internet connection. This allows citizens or authorities to connect to its localised access point to receive updates, especially if the main dashboard is compromised during the disaster.

Table 3.2.3 and figure 3.2.3 show the component breakdown and circuit design of this node respectively.

Node 3 – Alert Node				
Component Image	Component Name	Component Description	Role	Requirements
	ESP8266 Microcontroller	A WiFi enabled microcontroller	Display current environment status through text (LCD) and coloured LEDs,	3.3V Power source

			produce alarm sound during emergency warning. Host local dashboard	
	Buzzer	Produces buzzes/ sounds when it receives power	Produce an 'alarm' sound if extreme warning conditions exist	Libraries: • pitches.h – to control the 'buzz/ sound' the buzzer makes • NPN transistor • 1K resistor
	LCD Module Screen	A component that displays 16x2 characters digitally	Display the current condition status as well as any messages	Libraries: • LiquidCrystal_I2C – simplifies the 16x2 character LCD's with the I2C interface • Wire.h – native Arduino library for I2C communication
	LEDs (3x)	Light Emitting Diode – a component that produces light when a voltage is applied to it	Light up based on conditions. Red = critical, yellow = warning, green = okay	3x 220 resistors (one per LED)

Table 3.2.3 - Component breakdown for Node 3 (Alert Node)

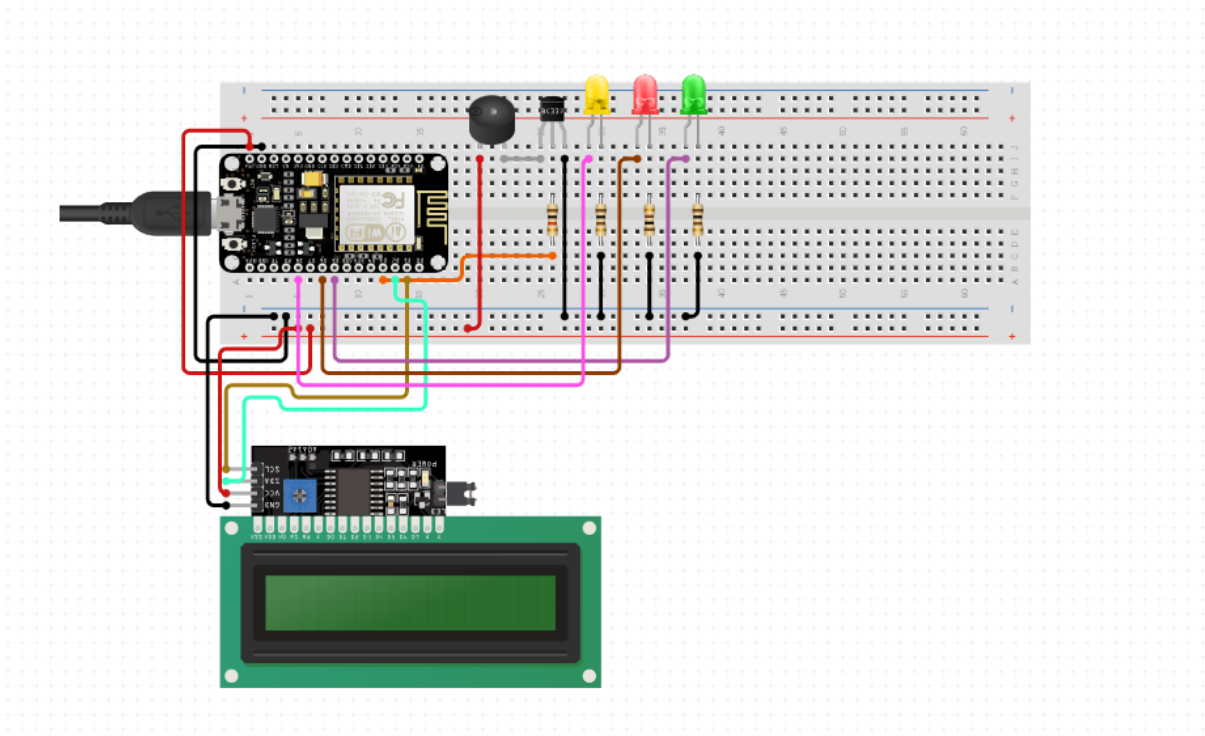


Figure 3.2.3 – *Circuit Diagram for Node 3 (alert node)*

Node-RED is used to create an administrative dashboard that acts as the primary interface for allowing authorities to define policies, monitor current system status and view history. It integrates a map API to visualise sensor locations and track geographical risk zones. The dashboard provides authorities/ system administrators control over the system by allowing them to override sensor reading intervals, force immediate sensor readings and control actions taken by the mitigation node. It also facilitates automated public emergency communication by broadcasting real-time SMS alerts to affected residents through an integrated Twilio API.

Figure 3.2.4 shows a data flow diagram between nodes, the dashboard and the communication direction between components within each node. This solution utilises a hybrid communication architecture using ESP-NOW as the primary communication method between nodes, and Wi-Fi to bridge the field nodes with the Node-Red dashboard. Since the Node-RED dashboard cannot be implemented on an ESP-8266, communication cannot depend upon ESP-NOW. Therefore, utilising the ESP8266's built in Wi-Fi module, the sensor node transmits the sensor and environment evaluation to the dashboard before entering its sleep state. The network also supports bi-directional communication, with the dashboard being able to send override instructions to both sensor and mitigation nodes, allowing system administrators to directly create force actions.

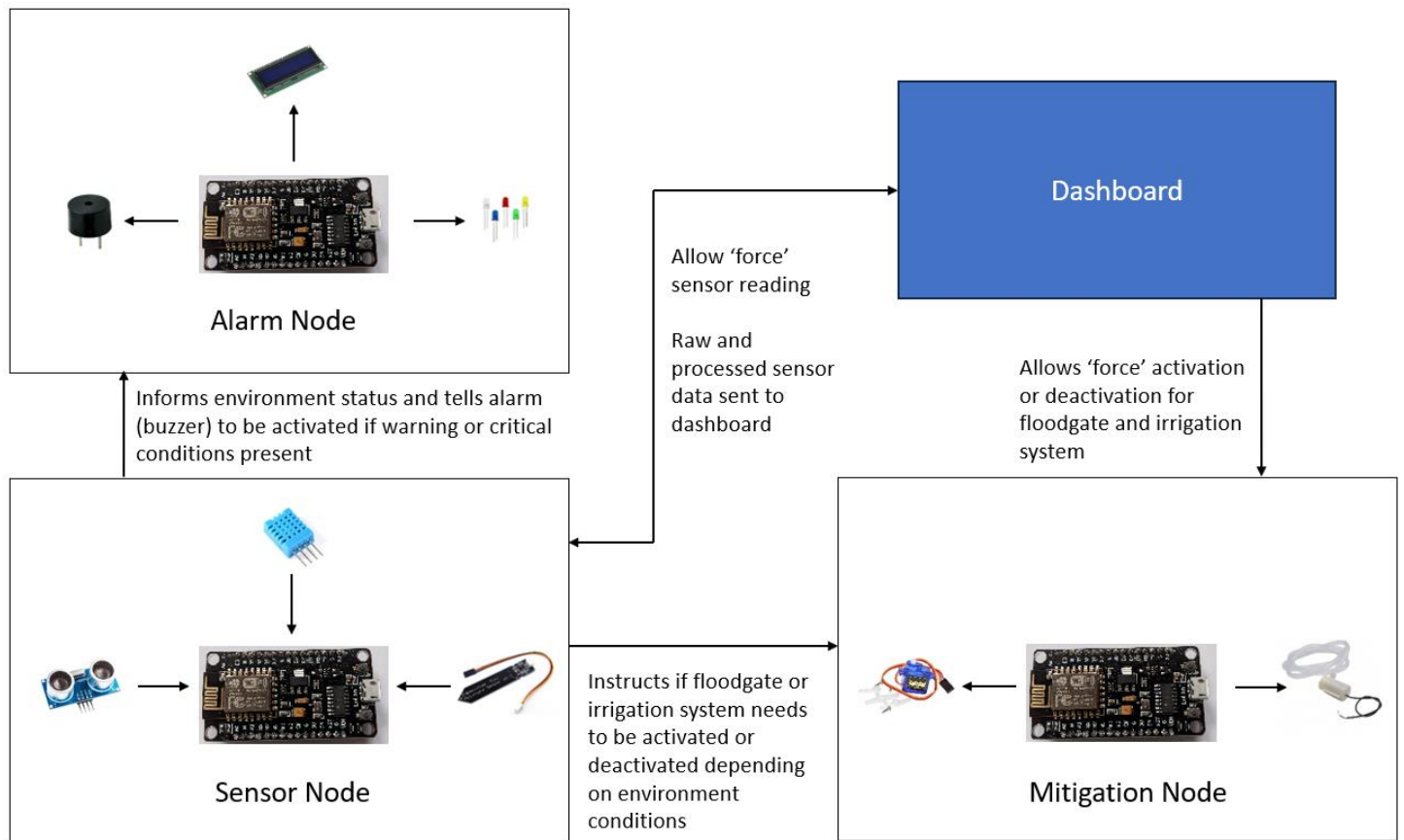


Figure 3.2.4 – data flow diagram between the nodes and the dashboard, communication direction between components and ESP8266 microcontroller within a node

3.3- Justification of Key Design Choices

Table 3.3.1 lists the key system features, and justifies them through the domain of the project and target users.

System Feature	Implementation	Domain Justification	Target User Justification
ESP8266	Using ESP8266 microcontroller for nodes 1, 2 and 3	• Low-cost, commercial availability • Integrated Wi-Fi module • Sleep state functionality for power-optimisation at remote, dangerous locations	• Resource-constrained developing countries/ districts – low cost, so lowers deployment cost and allows authorities to scale up and build dense solutions on a budget

Modular decentralised node separation	Three-tier hardware isolation, mapping nodes to sensor, mitigation and alert nodes	• Eliminates systematic points of failure – damage or failure of one node does not impact the rest, makes it more robust and resilient (enforces strict fault tolerance)	• Authorities – simplifies repairs, and allows scaling of the solution • Remote/ isolated residents – provides localised autonomous protection if cut off from emergency services
Peer-to-Peer mesh	Direct, router-free node-to-node communication using ESP-NOW protocol	• Removes dependency on external vulnerable communication grids (cellular towers, internet lines) that can easily collapse especially during natural disasters	• High-risk, disaster-prone residents and vulnerable remote communities – guarantees mitigation triggers execute in a timely manner, especially when every second counts
Edge Data Processing	Using fuzzy logic to make evaluations at the edge	• Provides a holistic and accurate overview and monitoring of current environmental conditions and how it will be affected. • Allows justified mitigation actions to be taken	• Authorities – able to quickly understand and view environment conditions, instead of manually evaluating complex raw sensor data
Adaptive Sleep Cycles	Sensor nodes operating in deep sleep loops dynamically scaled by calculated conditions	• Optimises power at remote, off-site monitoring lines	• Vulnerable remote communities – minimises physical maintenance, site visits and replacement frequency
Dual-tier dashboard/ interface	Main Node-RED dashboard, paired with local independent offline emergency web access points through alert nodes	• Decouples admin control from immediate safety-critical warning data • Crucial data and a form of connection is preserved if Node-RED dashboard is down or unable to connect	• High-risk resident demographics – homeowners can connect to the local web portal without Internet connection and receive critical updates • Authorities – have a separate access and control to the solution
External API integration	Integrating Weather API, Map API and the Twilio SMS API	• Combines live, local forecasts, allowing the fuzzy logic engine to make optimised and useful evaluations	• Local Municipiles and authorities – automatically considers local predictive weather intelligence, allows for

		• Allows tracking of sensors and current locations • Creates independent, automated digital broadcast loop to ensure instant wide-scale alerts without having a delay	better control and visualisation and instant communication • High-risk residents – broadcasted automated SMS alerts to residents before cellular towers possibly fail
--	--	--	--

Table 3.3.1 – A table justifying key proposed system features with regards to the project domain and target users

3.4- Work Plan

Figure 3.4.1 shows a Gantt chart for each phase of the solution implementation phase. Two weeks are left at the end for contingency, testing and writing the report will be implemented throughout the development. The work plan splits the development into phases, where at the end of each phase, a minimum viable product (MVP) is achieved.

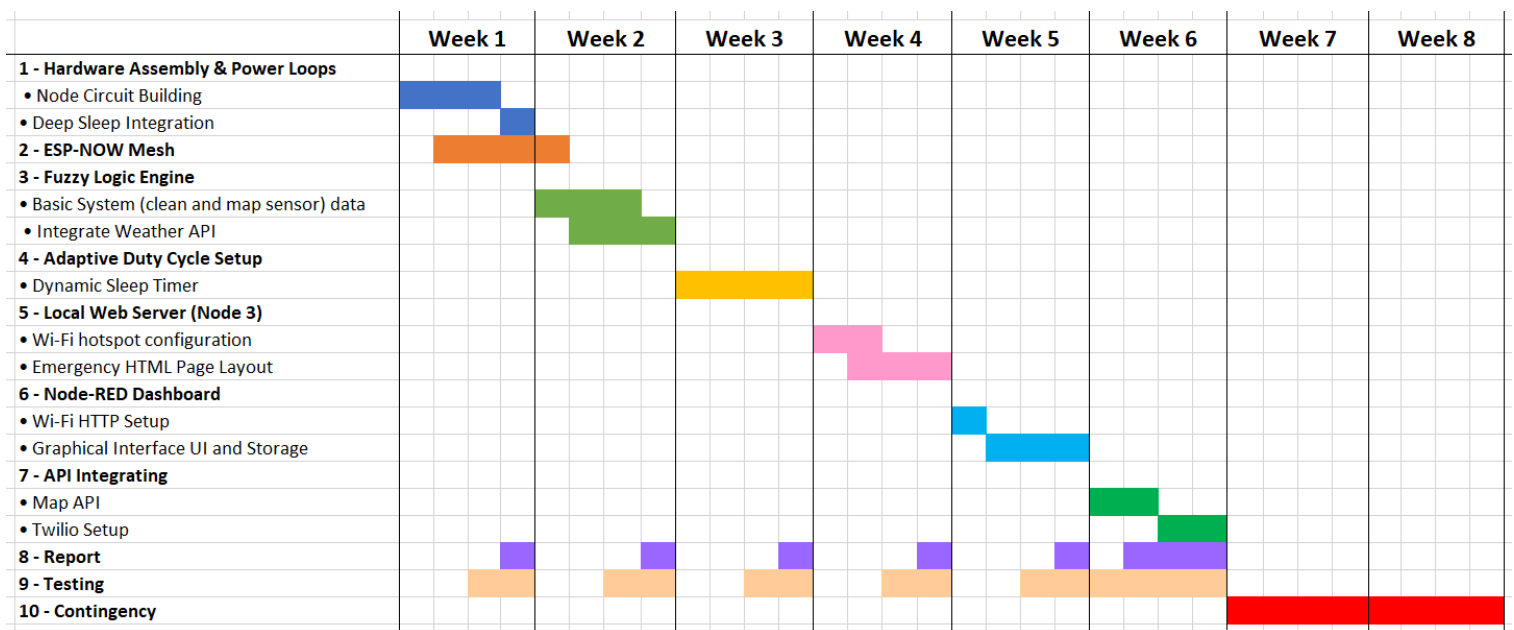


Figure 3.4.1 – Gantt Chart for the work plan to be implemented

Example risks to consider as part of the contingency plan include: faulty hardware (require time to order), sensor calibration issues, localised radio frequency interference causing ESP-NOW packets to be lost, and wireless connection timeouts.

3.5- Testing and Evaluation Plan

Due to the nature of this project, physical user testing is difficult. Therefore, the testing strategy will follow a Continuous Integration and Continuous Testing (CI/ CT) approach, inspired by Test-Driven Development (TTD). Prior to beginning a phase or week, test cases will be developed, which will later be used as an evaluation criteria at the end of the phase. This will allow for focused and prepared development, with the test cases also forming a plan. Four different areas will be covered during testing – hardware and integration, functional logic, web interface, and error handling. Table 3.5.1 shows examples of 10 test cases covering these areas.

Example Test Cases			
<u>Hardware and Integration Tests</u> <i>To ensure components are being handled correctly by the ESP8266 microcontrollers for each node</i>			
Test ID	Feature	Test Action	Expected Result
TC - 01	Sensor Node – Ultrasonic sensor (water level)	Fill the water tub with a lot more water (5 cm)	The dashboard shows a reading of 5cm increase in water level
TC - 02	Sensor Node - Deep Sleep Restart	Set the sleep timer in the code to 5 seconds	The node enters sleep mode, 5 seconds later turns on and takes a sensor reading as seen in dashboard, then returns to its sleep state
TC - 03	Mitigation Node – Floodgate Servo Activation	Make node 1 (sensor) send a message to node 2 (mitigation) to activate floodgate	The servo should rotate 90 degrees
<u>Functional Logic Tests</u> <i>To ensure the solution works as desired</i>			
TC-04	Fuzzy Logic Threshold Check	Manually type mock sensor values to mimic pre-drought conditions (e.g. low soil moisture, high temperature)	The fuzzy logic engine evaluates it as pre-drought conditions, activates irrigation system and updates

			the environment condition
TC - 05	Alarm Node Visual Switch	Change the current state value inside Node 3 to 'high risk'	Red LED should illuminate and buzzer should sound an alarm
Web Interface Testing			
<i>To ensure main dashboard and Local Access Points from Node 3 (Alert) work as intended</i>			
TC-06	Alert Node Local dashboard Page Access	Disconnect Internet access on device and enter the IP for the local dashboard	Typing in the IP should successfully open the main page for the dashboard
TC-07	Node-RED Dashboard Update	The sensor node takes a new reading, and transmits the data to the dashboard through Wi-Fi	The dashboard updates in real time and the new reading can be seen
Error Handling			
<i>To prevent undesirable results</i>			
TC-08	Lost Packet Signal Handling	Disconnect Node 2 and Node 3 from power source, and force Node 1 to transmit a data packet	Node 1 registers failed delivery flag, informs Node-RED dashboard and goes back to sleep
TC-09	Weather API dropout fallback	Type the Weather API address incorrectly, to simulate drop out	Node 1 registers connection failure, and immediately uses pre-saved data for its fuzzy logic engine
TC-10	Forced Mitigation Node Command	Click activate floodgate on Node-RED dashboard while system is in normal state	Node 2 (Mitigation) rotates servo 90 degrees upon receiving instruction

Table 5.3.1 – Example test cases and their expected results

4 – Feature Prototype

The prototype developed demonstrated the communication between two ESP8266 nodes using ESP-NOW to transmit and receive sensor readings from an ultrasonic sensor connected to node 1. Then, deep-sleep cycles were implemented such that node 1 would take readings from an ultrasonic sensor at 5 second intervals upon waking up from a deep sleep state.

To initialise ESP-NOW communication, the MAC address of node 2 (the receiver node) had to be taken, which was implemented through the code shown in figure 4.1.1. There was no need to take the MAC address of node 1, since one-way communication was implemented and it was acting as the sender node.

```
mac_address_finder.ino
1
2  #include <ESP8266WiFi.h>
3
4  void setup() {
5      // put your setup code here, to run once:
6      //start serial monitor
7      Serial.begin(9600);
8      delay(1000);
9      //get MAC address and print it to the serial mointor - only once
10     WiFi.mode(WIFI_STA);
11     Serial.print("Node 2 MAC Address is: ");
12     Serial.println(WiFi.macAddress());
13
14 }
15
16 void loop() {
17     // put your main code here, to run repeatedly:
18
19 }
20
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 0.9 (ESP-12 Module)' on 'COM3')

Node 2 MAC Address is: 34:94:54:95:AD:61

Figure 4.1.1 – Code and Serial Monitor output for identifying Node 2's (receiver) MAC address.

Once the ultrasonic sensor had taken a reading, it would transmit the data by first registering itself as a sender, then establishing a connection to node 2 using its IP address, it's role, channel and security key. If this wasn't established, then this was printed to the serial monitor and the node entered into its deep sleep state. If a connection was successful, then it broadcast a message to node 2 with the sensor reading data. Upon receiving the packet, node 2 would illuminate it's LED and print the received sensor reading to its serial monitor. Deep sleep states were implemented by connecting the microcontroller A0 pin to its RST pin. The code for taking the sensor reading and ESP-NOW transmission was

implemented in the setup function, since the ESP8266 microcontroller would ‘reset’, and therefore no looping function was necessary. Figure 4.1.2 and 4.1.3 show the serial monitor output for nodes 1 and 2 respectively.

```

r10l0r0$00n?l?b|??000?r0b?b00nn0l
--- Woken Up Successfully ---
Sensor Measured: 11.00 cm
Data package sent out.
Going to sleep for 5 seconds...
r10l0r0$00n?l?b|??000?r0b?b00nn0l
--- Woken Up Successfully ---
Sensor Measured: 11.00 cm
Data package sent out.
Going to sleep for 5 seconds...

```

Figure 4.1.2 – Serial Monitor output for node 1 (sensor sending node)

```

Data Received Distance: 0.00 cm
Data Received Distance: 45.46 cm
Data Received Distance: 29.73 cm
Data Received Distance: 0.00 cm
Data Received Distance: 43.06 cm
Data Received Distance: 56.71 cm
Data Received Distance: 2.11 cm
Data Received Distance: 0.00 cm
Data Received Distance: 11.00 cm

```

Figure 4.1.3 – Serial Monitor output for node 2 (receiving node)

Overall, the prototype works as intended to demonstrate ESP-NOW communication and deep sleep state cycles, however it was noticed that occasionally packets were lost (packets were sent by node 1, but not received by node 2). This is something to improve and consider to make the proposed IoT natural disaster monitoring solution robust and reliable.

References

- [1] World Meteorological Organization. 2026. Earth's climate swings increasingly out of balance. *World Meteorological Organization*. Retrieved from <https://wmo.int/news/media-centre/earths-climate-swings-increasingly-out-of-balance>
- [2] Elyse DeFranco. 2025. Floods, Droughts, Then Fires: Hydroclimate Whiplash is Speeding up Globally. *DRI*. Retrieved from <https://www.dri.edu/floods-droughts-then-fires-hydroclimate-whiplash-is-speeding-up-globally/>
- [3] Michele Metych. 2023. Natural Disaster | Causes, Types, & Facts | Britannica. *www.britannica.com*. Retrieved from <https://www.britannica.com/science/natural-disaster>
- [4] Awasthi Awasthi and Amin Salih Mohammed. 2020. IoT- A Technological Boon in Natural Disaster Prediction. *2019 6th International Conference on Computing for Sustainable Global Development (INDIACom)* (February 2020). Retrieved from <https://ieeexplore.ieee.org/document/8991244/>
- [5] Dinesh Goyal, Aditya Sharma, Kamal Deep Garg, Bhisham Sharma, and Imed Ben Dhaou. 2024. The Internet of Things (IoT) Contribution to Natural Disaster Management: Review. *2023 20th ACS/IEEE International Conference on Computer Systems and Applications (AICCSA)* (April 2024). Retrieved from <https://ieeexplore.ieee.org/document/10479272>
- [6] Haoming Xiang and Take Itagaki. 2025. The Social Impact of IoT in Disasters. (May 2025). Retrieved from <https://ieeexplore.ieee.org/document/11005074>
- [7] Yazan Ibrahim Alatoom and Omar Smadi. 2025. Embedded framework for low-cost pavement condition evaluation using microcontroller and single-board computer platforms. *Automation in Construction* 178, (October 2025), 106442. DOI:<https://doi.org/10.1016/j.autcon.2025.106442>
- [8] Raspberry Pi. Raspberry Pi Documentation - Raspberry Pi Hardware. *www.raspberrypi.com*. Retrieved from <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>
- [9] Yash Goyal. 2024. Comparative Study of Microcontoller: ARDUINO UNO, RASPBERRY PI 4, ESP 32. *International Journal for Research in Applied Science and Engineering Technology* 12, 7 (July 2024), 588–592. DOI:<https://doi.org/10.22214/ijraset.2024.63598>
- [10] Ali Abdelli, Wissem Boujnah, Younes Ben Salah, Khaled Issa, Hatem Garrab, Sarvar Hussain Nengroo, Mohsen Machhout, and Kyungsu Lee. 2025. Field Validated Hybrid ESP-NOW and Long Range IoT Monitoring System for Energy Autonomous Precision Agriculture. *IEEE Access* 13, (2025), 207511–207526. DOI:<https://doi.org/10.1109/access.2025.3641487>
- [11] Giulia De Masi. 2018. The impact of topology on Internet of Things: A multidisciplinary review. *2018 Advances in Science and Engineering Technology International Conferences (ASET)* (February 2018), 1–6. DOI:<https://doi.org/10.1109/icaset.2018.8376837>
- [12] Dania Eridani, Adian Fatchur Rochim, and Faiz Noerdiyan Cesara. 2021. Comparative Performance Study of ESP-NOW, Wi-Fi, Bluetooth Protocols based on Range, Transmission

Speed, Latency, Energy Usage and Barrier Resistance. *IEEE Xplore*, 322–328.
DOI:<https://doi.org/10.1109/iSemantic52711.2021.9573246>

- [13] Espressif Systems. 2026. ESP-NOW User Guide. *Espressif.com*. Retrieved from https://documentation.espressif.com/esp-now/master/User_Guide.md
- [14] LoRa Alliance. 2015. What is LoRaWAN®. *Lora-alliance.org*. Retrieved from <https://resources.lora-alliance.org/document/what-is-lorawan>
- [15] Ramakrishnan Raman and Syed Mohd Uzair Iqbal. 2024. IoT-based Flood Early Warning System for Effective Disaster Management. In *2024 International Conference on E-mobility, Power Control and Smart Systems (ICEMPS)*, IEEE (Institute of Electrical and Electronics Engineers). DOI:<https://doi.org/10.1109/icemps60684.2024.10559355>
- [16] Ahmed A. Radhi and Abdullahi A. Ibrahim. 2026. An intelligent IoT–machine learning framework for wildfire detection and prediction using a hybrid RF–XGB model. *Scientific Reports* (May 2026). DOI:<https://doi.org/10.1038/s41598-026-52395-w>
- [17] E Pragatheeswari, V Vishnu Priya, G Nisanth, and D Dhanushree. 2024. IoT Integrated Smart Rover System for Disaster Relief Management. In *2024 2nd International Conference on Sustainable Computing and Smart Systems (ICSCSS)*, IEEE (Institute of Electronics and Electrical Engineers). Retrieved from <https://ieeexplore.ieee.org/document/10625567>
DOI:<https://doi.org/10.1109/ICSCSS60660.2024.10625567>
- [18] Blynk. 2024. Introduction | Blynk Documentation. *Blynk.io*. Retrieved from <https://docs.blynk.io/en>